

UNIVERSIDAD TECNOLÓGICA NACIONAL · FACULTAD REGIONAL BUENOS AIRES

BiblioTech

Sistema de Gestión de Bibliotecas (SGB)



Entrega Final de Proyecto

Inteligencia Artificial Aplicada a Organizaciones

Integrantes

Tomás Bacchetta · Julio Flores · Matías Ledesma · Alex Raza

Buenos Aires, Argentina · Agosto 2026

BiblioTech (SGB) — Entrega Final

Inteligencia Artificial Aplicada a Organizaciones · UTN Facultad Regional Buenos Aires

Links

Repositorio GitHub
Aplicación web en producción
Video de demo
Otro recurso publicado

Parte 1 — El proyecto como aplicación real

Sección 1 · Presentación del equipo y del proyecto

Integrantes del grupo y rol de cada uno
Nombre del proyecto
Problema que resuelve
Público objetivo

Sección 2 · Arquitectura técnica

Diagrama de arquitectura general del sistema
Flujo de datos: entrada → salida
Componentes de IA vs. lógica tradicional
Ubicación de la memoria persistente
Diagrama de flujo de agentes
UML — casos de uso / secuencia / clases

Sección 3 · Stack tecnológico

Frontend
Backend
Base de datos
Modelo de IA
Orquestación
Despliegue

Sección 4 · Evidencia de funcionamiento

Capturas de pantalla del frontend (mínimo 3)
Video de demostración (opcional)
Log o registro de una sesión real

Sección 5 · Evaluación UX/UI

5.1 Heurísticas de Nielsen (mínimo 5 de 10)
5.2 Evaluación orientada al público objetivo

Sección 6 · Evaluación de ciberseguridad

Log de riesgos identificados (mínimo 4)

Sección 7 · IAs usadas en el co-work de desarrollo

Tabla: herramienta / para qué se usó / aporte
Reflexión obligatoria (1 párrafo)

Parte 2 — IA local en el proyecto

Pregunta 1 · Rol del LLM/SLM local en el proyecto

Pregunta 2 · Aporte al usuario de la aplicación

Pregunta 3 · Aporte a vos como profesional

Pregunta 4 · Limitaciones vs. API en la nube

Links

Enlaces públicos del proyecto

Repositorio GitHub:

- BackEnd: [BiblioTech-Back](#)
- FrontEnd: [BiblioTech-Front](#)
- Documentación: [BiblioTech-Informe](#)

Aplicación web en producción: <https://nicolasraza.github.io/TP-IA-UTN-BiblioTech-Front/>

Video de demo: *[pegar el link acá, u OMITIR si no graban video]*

Otro recurso publicado: Backend / API en producción —
<https://tp-ia-utn-bibliotech-back-production.up.railway.app/>

Sección 1 · Presentación del equipo y del proyecto

Integrantes y roles

Integrante	Rol principal	Responsabilidades
Tomás Bacchetta	Backend & Base de datos	Desarrollo del backend FastAPI y el modelo de datos (7 tablas, enums de PostgreSQL); implementación de los adaptadores entre los agentes de IA y la base real (repositorios_impl.py)
Julio Flores	Testing & QA · Documentación	Pruebas de extremo a extremo del sistema (backend, base de datos, frontend, integración); coordinación y redacción del informe final
Matías Ledesma	Integración Front-Back & Ciberseguridad	Liderazgo de la integración entre frontend y backend (PR que conectan la app Flutter a la API real) y configuración de la organización de GitHub (SSO, protección de ramas); revisión de pull requests generados con asistencia de IA (Claude, Jules) y seguridad y securización de la aplicación
Alex Raza	Frontend & CI/CD	Prototipo inicial de interfaz y coordinación de la migración a Flutter (Clean Architecture + BLoC); diseño y mantenimiento del pipeline CI/CD (tests automáticos, build web/APK, publicación en GitHub Pages y releases con versionado automático)

Nombre del proyecto

BiblioTech — Sistema de Gestión de Bibliotecas (SGB)

Problema que resuelve

La gestión bibliotecaria tradicional (alta de catálogo, préstamos, reservas, multas) implica tareas manuales y repetitivas: cargar datos de un libro nuevo, calcular vencimientos según el tipo de lector, gestionar colas de reserva y notificar disponibilidad. BiblioTech digitaliza este flujo de punta a punta y automatiza las tareas de mayor carga operativa mediante agentes de IA — captura de datos por OCR, enriquecimiento automático de fichas bibliográficas y planificación de reservas — reduciendo el trabajo manual del personal de biblioteca y mejorando la experiencia de los lectores.

Público objetivo

Bibliotecas (escolares, populares o institucionales) con tres perfiles de usuario diferenciados:

Lectores — categorías infantil, adolescente, adulto, docente e institucional

Bibliotecarios — responsables de la operación diaria (alta de catálogo, préstamos, devoluciones)

Administradores — visión global del sistema (indicadores, gestión de usuarios)

Sección 2 · Arquitectura técnica

Diagrama de arquitectura general del sistema

El siguiente diagrama muestra la arquitectura general de BiblioTech, cubriendo los tres aspectos que exige esta sección:

Flujo de datos (entrada → salida)

El Frontend envía las solicitudes del usuario al Backend (flecha "Entrada"), que las procesa y devuelve la respuesta (flecha "Salida"). Todo el tráfico pasa por una única API REST (FastAPI).

Componentes de IA vs. lógica tradicional

Dentro del Backend, el diagrama separa explícitamente dos módulos. El de "Lógica tradicional" (en azul) contiene la autenticación (JWT) y el CRUD de catálogo y circulación — reglas de negocio determinísticas, sin IA. El de "Agentes (IA)" (en púrpura) contiene los tres agentes que sí razonan sobre datos: Captura (OCR), Analizador (enriquecimiento bibliográfico) y Planificador (gestión de reservas) — estos son los únicos componentes que consultan servicios externos de IA/datos (Tesseract OCR, Google Books API).

Ubicación de la memoria persistente

La base de datos PostgreSQL (en verde, "Memoria persistente") vive fuera de ambos módulos, y ambos —tanto la lógica tradicional como los agentes— leen y escriben sobre ella por igual (flechas "Lectura/escritura"). No hay una base de datos separada para los agentes: toda la información del sistema converge en un único lugar.

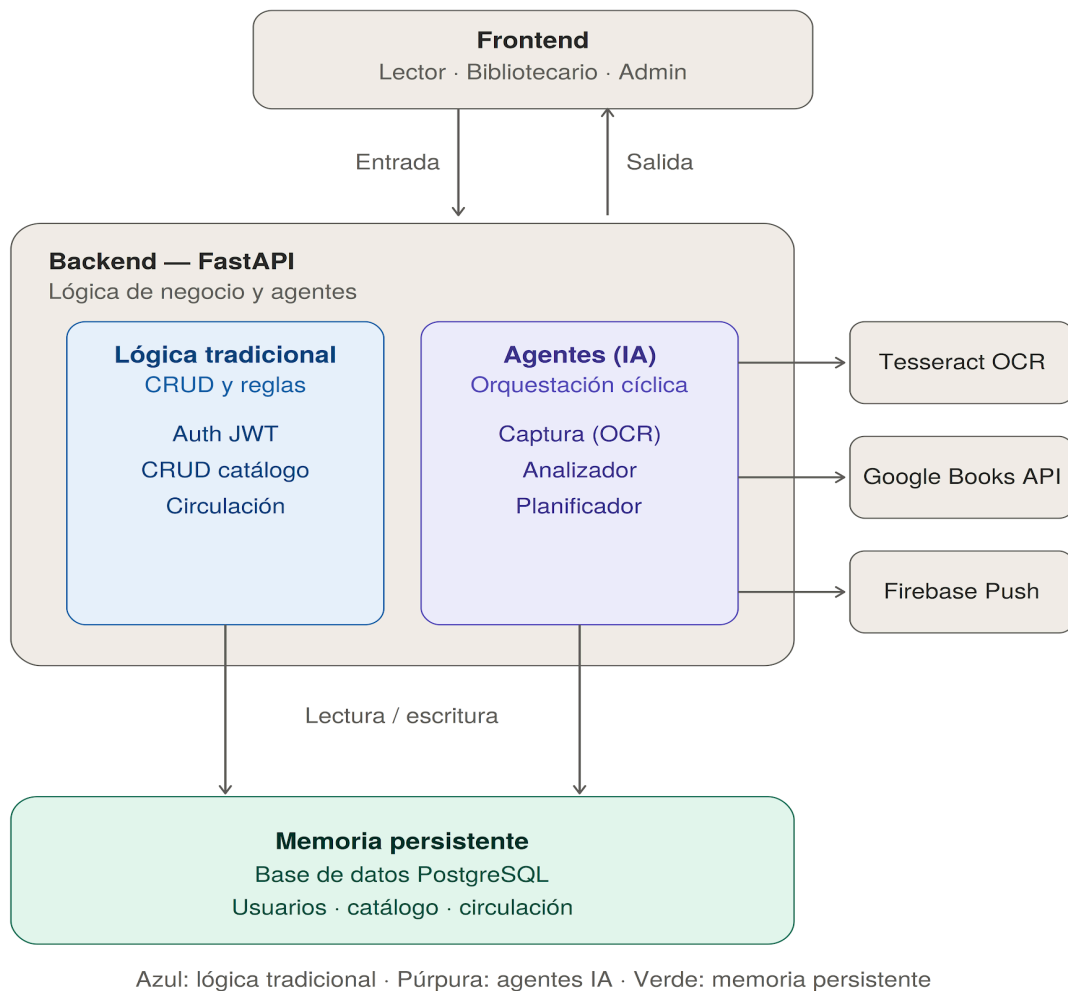


Diagrama entidad-relación (ERD) — Base de datos

Modelo de datos completo de BiblioTech: siete entidades (Usuarios, Lectores, Títulos, Ejemplares, Préstamos, Reservas y Multas) con sus relaciones y claves. USUARIOS distingue el rol de cada cuenta y puede dar de alta títulos o registrar préstamos; TÍTULOS representa la obra bibliográfica, separada de EJEMPLARES (las copias físicas con QR propio). Diagrama completo en el Anexo Técnico, Figura 1.

Diagrama de flujo de agentes

El sistema sigue el ciclo de orquestación cíclica definido en el diseño conceptual (TP1): Observación → Análisis → Planificación → Acción → Evaluación → Aprendizaje. La implementación real fija el orden Observación → Análisis → Decisión → Acción en un único punto del código (CorrerCicloDeAgentes), con una separación garantizada por diseño: el Agente Evaluador es puro y solo decide, sin acceso a nada que pueda modificar; el Agente Planificador solo ejecuta las decisiones recibidas, sin volver a evaluar. El Agente de Aprendizaje corre aparte, de forma asíncrona, cada vez que hay una corrección manual o un clic sobre una recomendación — no como un paso fijo de cada ciclo. Diagrama completo en el Anexo Técnico, Figura 2.

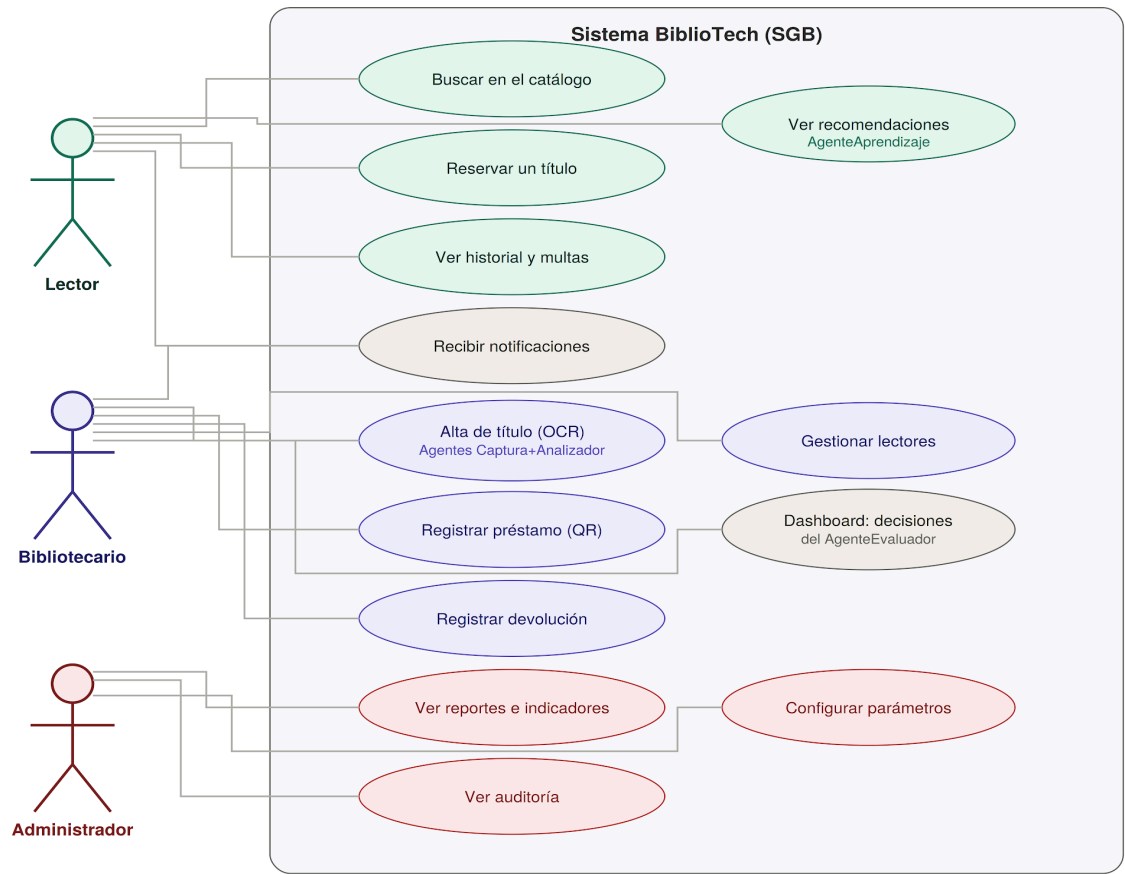
La separación "Evaluador decide / Planificador ejecuta" está garantizada por construcción, no por convención: el Planificador no tiene acceso al estado del sistema, solo a la lista de decisiones ya tomadas.

UML

Se documentan los tres diagramas: casos de uso, secuencia y clases.

1 · Diagrama de casos de uso

Casos de uso principales por rol, contruidos a partir de las pantallas relevadas en la Sección 4 y de los endpoints documentados en el Anexo. En gris, los casos de uso compartidos entre roles.



2 · Diagrama de secuencia

Se eligió modelar la devolución de un ejemplar con una reserva pendiente en cola, por ser la interacción donde más se nota la diferencia entre lógica tradicional y agente de IA: la devolución en sí es CRUD, pero a partir de ahí el Agente Planificador decide solo a quién asignarle el ejemplar liberado y dispara la notificación, sin que ningún humano lo pida explícitamente. Diagrama completo en el Anexo Técnico, Figura 3.

3 · Diagrama de clases

Versión con notación UML (atributos tipados, visibilidad y multiplicidad) del mismo modelo de datos que el ERD de esta sección. Las multiplicidades (1, 0..1, 0..*) se derivan de las constraints reales relevadas en la base de producción (NOT NULL, nullable, UNIQUE) — no son estimadas. Diagrama completo en el Anexo Técnico, Figura 4.

Sección 3 · Stack tecnológico

Componente	Tecnología / Herramienta	Por qué se eligió esta y no otra
Frontend	Flutter (Dart), compilado a Web	Un solo código fuente para las tres vistas de rol (lector, bibliotecario, administrador) y potencial de compilar a Android/iOS a futuro sin reescribir. [COMPLETAR: confirmar con Alex/Matías si se evaluó alguna alternativa antes de elegir Flutter.]
Backend	Python 3.11 + FastAPI, SQLAlchemy async (asyncpg)	Tipado con Pydantic, documentación automática (Swagger/OpenAPI) y soporte nativo async, clave para no bloquear el servidor mientras los agentes hacen llamadas externas (OCR, Google Books, Ollama). 39 endpoints organizados en 7 dominios (ver Anexo).
Base de datos	PostgreSQL, alojada en Railway	Relacional, con siete entidades bien relacionadas (usuarios, lectores, títulos, ejemplares, préstamos, reservas, multas — ver ERD en Sección 2) y enums nativos de Postgres para controlar la integridad de los estados, algo que un almacenamiento documental (Firebase/Mongo) resolvería de forma menos estricta.
Modelo de IA	Ollama (modelo local, opcional/activable)	Se eligió correr el modelo local (vía GET /dashboard/resumen-ia) en vez de una API en la nube para que los datos operativos de la biblioteca no salgan de la infraestructura propia, sin costo por token, aceptando a cambio menor calidad de redacción (ver Parte 2, Pregunta 4). [COMPLETAR: nombre y tamaño exacto del modelo servido — ej. Llama 3.2 3B, Gemma 2.]
Orquestación	Código propio (5 agentes en app/agents/), sin framework externo	El ciclo de 6 etapas (Observación → Análisis → Planificación → Acción → Evaluación → Aprendizaje, ver diagrama en Sección 2) es simple y determinístico, sin necesidad de la complejidad de LangChain/n8n para un flujo con 5 agentes de responsabilidad acotada. Agente de Captura (OCR) · Agente Analizador (enriquecimiento) · Agente Planificador (cola de reservas + notificaciones, vía APScheduler y Firebase) · AgenteAprendizaje (recomendaciones 70% historial / 30% popularidad) · AgenteEvaluador (resumen ejecutivo con Ollama).
Despliegue	Railway (backend + base de datos)	Backend y base de datos en el mismo proyecto, con deploy automático en cada push a GitHub — evaluado también contra la combinación Neon + Render, descartada por requerir coordinar dos servicios separados. Verificado funcionando (curl al health check, y flujo completo de préstamo/devolución probado vía Postman — ver Sección 4). [COMPLETAR: dónde se publica el frontend Flutter — se sabe que hay CI hacia GitHub Pages, falta confirmar la URL pública.]

Sección 4 · Evidencia de funcionamiento

Capturas relevadas el 19/08/2026 sobre la app Flutter ejecutada localmente. Ver nota de transparencia al final de esta sección sobre qué partes ya están conectadas al backend de producción.

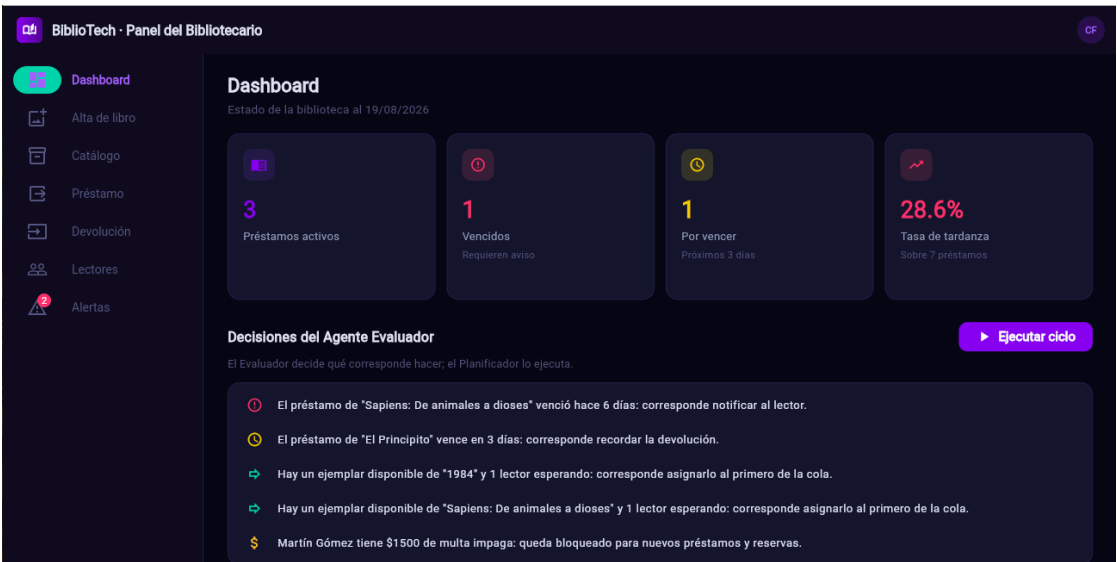
Capturas de pantalla del frontend

Capturas tomadas ejecutando la app Flutter en modo web (flutter run -d chrome), sobre las cuatro pantallas principales del sistema, una por rol más la pantalla de acceso.



Pantalla de acceso — selección de rol (Lector / Bibliotecario / Administrador)

Capturas adicionales (portal del Lector y panel de Administración) en el Anexo Técnico, Figuras 5 y 6.



Panel del Bibliotecario — dashboard con las decisiones del Agente Evaluador en lenguaje natural

Log o registro de una sesión real

Verificación de backend en producción (Railway) — 19/08/2026:

```
$ curl -i https://tp-ia-utn-bibliotech-back-production.up.railway.app/

HTTP/2 200 · content-type: application/json · {"status":"ok","sistema":"SGB v1.0"}
```

Flujo completo de sesión real (vía Postman, contra producción)

Se ejecutó de punta a punta el ciclo completo de circulación —login, alta de lector, alta de título y ejemplar, préstamo y devolución— contra el backend real en Railway, usando la colección de Postman generada desde el OpenAPI del backend. A continuación, cada paso con su request y la respuesta real obtenida.

1. Login

```
POST /api/v1/auth/login {"email":"admin@biblioteca.com","password":"admin123"}
200 OK → access_token (JWT) · rol: "administrador"
```

2. Alta de lector

```
POST /api/v1/lectores/
{"nombre":"Julio","apellido":"Flores","documento":"12345678","categoria":"adulto",
...}
201 Created → lector id: 1, estado: "activo", usuario_id: 2
```

3. Alta de ejemplar (sobre título ya existente en el catálogo)

```
POST /api/v1/catalogo/ejemplares
{"titulo_id":1,"condicion":"nuevo","ubicacion_fisica":"Estante B2"}
201 Created → ejemplar id: 1, codigo_qr: "SGB-C7633217862D", estado: "disponible"
```

4. Préstamo

```
POST /api/v1/prestamos {"qr_ejemplar":"SGB-C7633217862D","lector_id":1}
201 Created → préstamo id: 1, estado: "activo", vence 2026-09-02 (14 días — regla
de categoría "adulto" aplicada correctamente)
```

5. Devolución

```
POST /api/v1/prestamos/devolucion {"qr_ejemplar":"SGB-C7633217862D"}
200 OK → estado: "devuelto", fecha_devolucion_real: 2026-08-19 (mismo día, sin
multa)
```

Hallazgos durante la prueba

- GET /api/v1/auth/me devuelve 500 (Internal Server Error) en producción — mismo bug ya diagnosticado en el entorno local: falta eager loading (selectinload) de la relación usuario→lector en la consulta de deps.py, lo que dispara sqlalchemy.exc.MissingGreenlet al serializar la respuesta. Pendiente de corrección.
- La respuesta de POST /prestamos/devolucion trae titulo_id: null, mientras que el préstamo original sí lo incluía — posible inconsistencia menor en el serializer de ese endpoint, sin impacto funcional observado.
- Algunos registros de catálogo cargados previamente por el equipo quedaron con valores de placeholder sin completar (ej. "idioma": "string") — conviene revisar y limpiar los datos de prueba en producción antes de la demo final.

Nota de transparencia: este flujo se probó directamente contra la API (vía Postman), no navegando el frontend Flutter. Al momento de las capturas de la Sección 4, el frontend en modo local solo tenía conectado a este backend real el módulo de Autenticación; el resto de las pantallas (catálogo, dashboard, reportes) mostraban datos de un datasource local de demostración, como indica el aviso "Entorno de demostración" visible en la pantalla de acceso. [COMPLETAR: confirmar si para la entrega final el equipo conecta también catálogo/circulación/agentes al backend de producción desde el frontend, o si se documenta esta limitación tal cual.]

Video de demostración

Opcional según el índice — [COMPLETAR si el equipo decide grabarlo]

Sección 5 · Evaluación UX/UI

5.1 Heurísticas de Nielsen (mínimo 5 de 10)

Evaluación sobre las 4 pantallas relevadas (acceso, portal del lector, panel del bibliotecario y panel de administración), ejecutando la app Flutter localmente:

Se evaluaron 6 heurísticas con evidencia concreta de las capturas: el sistema cumple 5 de forma clara — visibilidad del estado (contadores en tiempo real, disponibilidad por título), coincidencia con el mundo real (patrón de tienda online, lenguaje llano del Agente Evaluador), consistencia visual entre roles, reconocimiento antes que recuerdo (decisiones en frases accionables, no en tablas crudas) y diseño minimalista. Una heurística — ayuda para reconocer y resolver errores — queda marcada como no evaluada, ya que las capturas relevadas cubren únicamente estados normales de la aplicación, sin forzar casos de error. Tabla completa con la evidencia y observación detallada de cada heurística en el Anexo Técnico.

5.2 Evaluación orientada al público objetivo

Lectores: el catálogo (portadas, filtros por género, buscador) sigue un patrón familiar para cualquier adulto o adolescente acostumbrado a apps de compras o streaming, por lo que el aprendizaje inicial debería ser bajo para esos segmentos. No se evaluó específicamente la experiencia para lectores infantiles ni para el flujo con tutor (que vimos en el modelo de datos — campos tutor_nombre/tutor_telefono), ya que la pantalla relevada corresponde al portal de lector genérico; queda pendiente revisar si existe una variante de interfaz simplificada para ese segmento.

Bibliotecarios: es el rol mejor resuelto en términos de UX orientada a la tarea. El dashboard antepone lo urgente (préstamos vencidos, por vencer) a lo informativo, y el bloque "Decisiones del Agente Evaluador" convierte el trabajo de interpretación de datos —que en un sistema tradicional recaería en el bibliotecario— en una lista de acciones ya resueltas, reduciendo la carga cognitiva de alguien que probablemente atiende el mostrador y el sistema al mismo tiempo.

Administradores: el panel de Reportes prioriza indicadores agregados y una tendencia temporal (gráfico de préstamos por mes, total vs. tardíos) antes que el detalle operativo del día a día, coherente con una visión gerencial del sistema. El menú lateral (Parámetros, Auditoría, Aprendizaje, Sugerencias) sugiere que la configuración fina del sistema vive en este rol, separada de la operación diaria del bibliotecario — buena separación de responsabilidades a nivel de interfaz.

[COMPLETAR: esta evaluación se hizo sobre la app corriendo localmente con datos de demostración (ver nota de transparencia en Sección 4); conviene repetirla, aunque sea brevemente, contra la versión publicada en producción antes de la entrega.]

Sección 6 · Evaluación de ciberseguridad

Riesgo identificado	Tipo (OWASP / privacidad / acceso)	Medida implementada o decisión tomada
API key de Google Books hardcodeada como valor default en config.py, expuesta en el historial de Git	Secretos en código	Pendiente de remediación: revocar la key actual en Google Cloud Console y moverla a variable de entorno (.env), que ya está correctamente excluida por .gitignore
Acceso indebido a la gestión de cuentas de staff (bibliotecario/administrador)	Control de acceso	Los endpoints /usuarios/* están scoped exclusivamente a roles bibliotecario/administrador — verificado empíricamente: devuelven 404 al intentar operar sobre un usuario con rol lector
Exposición de contraseñas de usuario	Autenticación	Hashing con bcrypt/passlib (nunca texto plano); sesión con JWT de expiración configurable (ACCESS_TOKEN_EXPIRE_MINUTES)
Configuración incorrecta de CORS bloqueando/exponiendo la API	OWASP A05:2021 — Security Misconfiguration	Detectado y corregido durante el desarrollo (rama claude/fix-me-500-cors, mergeada a main)
Datos personales de lectores menores de edad	Privacidad	Campo consentimiento_datos obligatorio en el alta de lector; datos de tutor (tutor_nombre, tutor_telefono) separados y asociados a la categoría del lector
Inyección de prompt en el resumen ejecutivo del AgenteEvaluador (único agente que usa un LLM)	Prompt injection	El resumen se genera a partir de indicadores ya calculados por lógica determinística (préstamos activos, tasas, etc.), no de texto libre ingresado por el usuario — acota la superficie de inyección
Cambios directos a main sin revisión, código no probado llegando a producción (ocurrió varias veces durante el desarrollo)	Control de acceso / Integridad del código	Branch protection rules activadas en ambos repositorios: Pull Request obligatorio antes de mergear a main

Sección 7 · IAs usadas en el co-work de desarrollo

Herramienta	Para qué se usó	Aporte
Claude (Anthropic)	Resolución de errores de deploy en Railway (ConnectionRefusedError por variable de entorno mal referenciada), evaluación de opciones de hosting (Railway vs. Render vs. Neon), redacción y estructuración de la documentación final	Aceleró el diagnóstico de problemas de infraestructura y liberó tiempo del equipo para enfocarse en el desarrollo del backend y frontend en la recta final
[COMPLETAR si usaron otra]		

Reflexión obligatoria (1 párrafo)

[COMPLETAR — reflexión genuina del equipo. Sugerencia: qué tareas delegaron a IA vs. cuáles hicieron ustedes mismos, si notaron algún error o alucinación que tuvieron que corregir, y si el uso de estas herramientas cambió el ritmo o calidad del trabajo final]

Parte 2 — IA local en el proyecto

Pregunta 1 · Rol del LLM/SLM local en el proyecto

En BiblioTech, un modelo local servido vía Ollama cumple un rol acotado y opcional: el AgenteEvaluador lo usa para convertir los indicadores operativos crudos del dashboard (préstamos activos, vencidos, reservas en cola, tasa de devolución en tiempo, títulos y lectores más activos) en un resumen ejecutivo en lenguaje natural para el administrador. Es una capa de interpretación sobre datos ya calculados —no reemplaza el cálculo de esos indicadores, que sigue siendo lógica determinística de negocio— y por eso está diseñado como opcional: el dashboard funciona igual sin el modelo, solo que sin el resumen narrativo.

[COMPLETAR: si además usan el modelo en el Agente Analizador para interpretar el texto del OCR, sumarlo acá — el Swagger solo confirma su uso en el resumen-ia]

Pregunta 2 · Aporte al usuario de la aplicación

Para el bibliotecario, el aporte central es la reducción del tiempo de alta de catálogo: en vez de tipear manualmente los datos de cada libro nuevo, el Agente de Captura + Analizador convierte tres fotos (tapa, contratapa, ficha) en una ficha bibliográfica pre-completada que solo requiere validación humana —ninguna publicación al catálogo ocurre sin confirmación explícita del bibliotecario, aunque la IA tenga alta confianza en el resultado. Para el lector, el AgenteAprendizaje personaliza las recomendaciones según su historial, y el Agente Planificador gestiona automáticamente la cola de reservas y notifica disponibilidad sin intervención manual del personal.

Pregunta 3 · Aporte a vos como profesional

[COMPLETAR con una reflexión personal genuina — esta pregunta pide tu experiencia, no puede completarse por vos. Pensá en: qué aprendiste integrando agentes de IA a un flujo de backend tradicional, qué desafíos tuvo, y cómo cambia tu forma de diseñar sistemas de ahora en más]

Pregunta 4 · Limitaciones vs. API en la nube

Al ser Ollama un modelo local y de menor tamaño que los modelos grandes en la nube (GPT, Claude, Gemini), la calidad de redacción del resumen ejecutivo es más limitada y puede requerir prompts más específicos para mantener consistencia. También consume recursos de cómputo del propio servidor de Railway donde corre el backend, a diferencia de delegar ese procesamiento a una API externa. La contrapartida es que no depende de conectividad ni de costo por token, y los datos operativos de la biblioteca no salen de la infraestructura propia. Por eso el equipo lo dejó como funcionalidad opcional: el sistema es completamente operativo sin él.

[COMPLETAR: si notaron en la práctica alguna limitación concreta corriendo Ollama en Railway — ej. tiempo de respuesta, si el plan de Railway alcanza en recursos]

Entregable opcional · Demo con Ollama

[COMPLETAR si el equipo decide incluir este entregable opcional]

Anexo Técnico

La referencia completa de los 39 endpoints de la API (agrupados por dominio: Autenticación, Usuarios, Lectores, Catálogo, Circulación, Dashboard y Health), los diagramas a tamaño completo y la tabla íntegra de heurísticas de Nielsen están en el documento adjunto "Anexo Técnico — BiblioTech", entregado junto con este informe.